# Alex Rivera

alex.rivera@example.com
+1 (555) 014-2288
github.com/example
Bristol, UK

Backend & Distributed Systems Engineer — projecting toward Staff Platform Engineer

**PROJECTION** — TARGET STATE 1 MARCH 2027 NOT A RECORD OF EXPERIENCE

## SUMMARY

Backend engineer with six years on high-throughput payment infrastructure, moving deliberately toward platform and reliability ownership. The lines below marked *Projected* are not yet true; each one is earned by a dated Milestone in the accompanying roadmap, and each Milestone names third-party-checkable Evidence.

## EXPERIENCE

**Senior Software Engineer**, Meridian Payments — Mar 2022 — Mar 2027

- Owned the settlement ledger service handling 4.2M transactions per day at p99 < 90 ms.
- Led the migration of the ledger from a single Postgres primary to a sharded topology, cutting peak write contention by 61% – framed for platform ownership rather than feature delivery. REFRAMED
- Built the idempotency layer that eliminated duplicate-capture incidents entirely across two peak seasons.
- Published *meridian-shardkit*, an open-source Postgres resharding toolkit, and drove it to first external production adopter. PROJECTED
- Ran the service's first formal error-budget policy, with SLOs published and reviewed monthly by the platform group. PROJECTED
- Mentored three engineers through their first on-call rotations; wrote the runbook set still in use.

**Software Engineer**, Northgate Logistics — Jul 2019 — Feb 2022

- Rewrote the route-optimisation batch pipeline in Go, taking a nightly 6-hour job to 38 minutes.
- Designed the event schema that eleven downstream teams consumed – restated as interface stewardship, the scope Staff roles are judged on. REFRAMED
- Introduced contract tests between the dispatch and billing services, ending a recurring class of release rollbacks.
- Ran the incident review process for the logistics group for eighteen months.

**Junior Engineer**, Fenwick Analytics — Sep 2018 — Jun 2019

- Maintained the ingestion connectors for eight third-party data vendors.
- Cut the nightly ETL failure rate from roughly one in four runs to under one in fifty.

## OPEN SOURCE & WRITING

- Merged three contributions to the Vitess resharding path, including a fix to the vreplication catch-up loop. PROJECTED
- Wrote a five-part series on online resharding without downtime; the first part reached the front page of Lobsters. PROJECTED
- Gave a 25-minute talk on error-budget policy at a regional SRE meetup, recording published. PROJECTED
- Long-standing maintainer of *pgbench-scenarios*, a small load-shape library with 340 stars.

## SKILLS

| | |
|---|---|
| LANGUAGES | Go, Python, SQL, TypeScript, a working amount of Rust |
| DATA | PostgreSQL (incl. logical replication, partitioning), Vitess, Redis, Kafka, ClickHouse |
| PLATFORM | Kubernetes, Terraform, ArgoCD, GitHub Actions, Nix for dev shells |
| OBSERVABILITY | Prometheus, Grafana, OpenTelemetry, error-budget and SLO practice |
| PRACTICE | Incident review, capacity planning, design review facilitation, on-call rotation design |

## CERTIFICATIONS

- Certified Kubernetes Administrator (CKA), certification id published on the Linux Foundation directory. PROJECTED
- AWS Certified Solutions Architect – Professional. PROJECTED
- PostgreSQL 14 Associate (EDB), 2023.

## SELECTED PROJECTS

**meridian-shardkit** 2026—27

- Online resharding toolkit for Postgres: cutover orchestration, dual-write verification, and a rollback path that has been exercised in anger. PROJECTED
- Documented against a public benchmark suite so adopters can reproduce the numbers. PROJECTED

**pgbench-scenarios** 2021—

- A library of realistic load shapes for Postgres benchmarking; used in two published papers.
- Maintained with a monthly release cadence and a documented compatibility policy.

**ledger-fuzz** 2024—

- Property-based fuzzer for double-entry ledger invariants; found four latent bugs in internal services.

**runbook-lint** 2023—

- Static checks for operational runbooks: every step must name a verification and a rollback.

## LONG BLOCK — DELIBERATELY SPANS A PAGE BOUNDARY

**Break-control probe**, every item below carries break-inside: avoid page 2 → 3

- Probe line 01 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 02 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 03 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 04 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 05 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 06 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 07 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 08 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 09 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 10 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 11 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 12 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 13 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.
- Probe line 14 – a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.

- Probe line 15 — a paragraph long enough to be split by a naive layout engine, so that the presence or absence of a mid-item break is observable in the rendered PDF without measuring anything by hand. If break-inside is honoured, this item appears whole on one page.

> **Second background-colour probe.** This block sits deep in the document, so it also tests whether background colour survives on pages after the first. If this box is white in the PDF, background printing is off.

## STAMP REPEAT PROBE

**PROJECTION** — TARGET STATE 1 MARCH 2027 NOT A RECORD OF EXPERIENCE

A second Stamp band, far from the top of the document, so that background colour on a later page is unambiguous.